Universidad Nacional de Tucumán

Facultad de Ciencias Exactas y Tecnología



Simulación y paralelización de caminatas aleatorias (Random Walk) mediante MPI

Diseño del algoritmo, automatización experimental y evaluación de escalabilidad

Por: Rodriguez Bernal Javier Alejandro, Rollan Natalia, Salcedo Cristian Facundo

2026

Métodos numéricos II

Departamento Ciencias de Computación

Resumen

Se desarrolló en C, mediante MPI, un simulador de caminatas aleatorias para los casos 1D y 2D solicitados en la consigna. Como ampliación del trabajo, se implementó una versión parametrizable en D dimensiones y se ejecutaron también pruebas en 3D y 4D. Con simulación masiva (método de Monte Carlo) se verificó la distancia promedio al origen y la forma de la distribución de la posición final, además se midió cómo escala el programa al aumentar la cantidad de procesos. Cada proceso simula caminatas completas e independientes con su propio flujo del generador PCG32 y solo se comunica al final, para sumar las distancias.

La distancia promedio simulada coincidió con la fórmula teórica: con un millón de caminatas y entre 100 y 5000 pasos, el error relativo máximo fue de 0,157 %, y los histogramas de la posición final tienen la forma esperada. La escalabilidad se midió con 10 millones de caminatas de 1000 pasos, con la mediana de cinco repeticiones y con todos los números de procesos consecutivos, en cuatro sistemas: un clúster (hasta 12 procesos), una PC de 6 núcleos, una netbook de 4 núcleos y una PC de 2 núcleos físicos con 4 hilos.

En el clúster, con 12 procesos, el speedup fue 11,978 en 1D y 11,996 en 2D, con eficiencias de 99,82 % y 99,97 %, respectivamente. En los equipos personales aparecieron diferencias asociadas al número de núcleos físicos, los hilos lógicos y las condiciones de ejecución. Se detectó también que la implementación especializada de 2D tiene un costo por paso notablemente superior al de la versión general.

Alcance: 1D y 2D constituyen el núcleo de la evaluación. 3D y 4D se presentan de forma sintética como extensión funcional y experimental del equipo.

Índice

1. Introducción

1.1 Fundamento teórico

1.2 Objetivos

2. Datos, implementación y metodología

2.1 Configuración de los experimentos

2.2 Estrategia de paralelización

2.3 Implementación MPI y generación aleatoria

2.4 Medición y reproducibilidad

3. Resultados

3.1 Validación de 1D y 2D

3.2 Distribución de posiciones finales

3.3 Escalabilidad en el clúster

3.4 Comparación entre los cuatro sistemas

3.5 Tiempo absoluto y anomalía de 2D

3.6 PC de dos núcleos: hilos lógicos

3.7 Extensión a 3D y 4D

4. Conclusiones

4.1 Limitaciones

4.2 Mejoras recomendadas

5. Referencias

1. Introducción

La caminata aleatoria (random walk) es un modelo sencillo de difusión: una partícula parte del origen y, a cada paso, elige aleatoriamente uno de los movimientos permitidos. Aunque no pueda predecirse el punto final de una trayectoria particular, una cantidad grande de simulaciones permite observar regularidades estadísticas: la distancia promedio al origen crece como la raíz cuadrada de la cantidad de pasos y la posición final se reparte en forma de campana.

Este trabajo desarrolla en C, con la librería de paso de mensajes MPI, un simulador paralelo de caminatas aleatorias. Resuelve los casos pedidos por la consigna (1D y 2D) y los generaliza a D dimensiones, de modo que también se probaron 3D y 4D. Además de comparar los resultados con la teoría, se evalúa cómo se comporta el programa al paralelizarse en cuatro sistemas de cómputo distintos.

1.1 Fundamento teórico

Una caminata de M pasos parte del origen y en cada paso se desplaza una unidad en una dirección elegida al azar. En 1D el paso es +1 o − 1 con probabilidad 1/2; en 2D se elige con igual probabilidad entre los cuatro movimientos sobre los ejes (derecha, izquierda, arriba, abajo); en D dimensiones hay 2*D direcciones posibles y en cada paso cambia una sola coordenada. Al terminar los M pasos se calcula la distancia final al origen, R, y se simulan N caminatas independientes. La distancia promedio simulada es:

$$\bar{R} = (R_1 + R_2 + \ldots + R_N) / N \qquad (1)$$

Como los desplazamientos son independientes y de longitud unitaria, se cumple exactamente $E[R^2] = M$. Para M grande, la posición final se aproxima por una distribución normal en cada coordenada, de varianza M/D.

Caso	Distancia promedio teórica (M grande)	
1D	$E[R] \approx \sqrt{(2M/\pi)}$	(2)
2D	$E[R] \approx \sqrt{(\pi M)} / 2$	(3)
D dimensiones	$E[R] \approx \sqrt{(2M/D)} \cdot \Gamma((D + 1)/2) / \Gamma(D/2)$	(4)

La función Γ es la función gamma. La fórmula general incluye los casos 1D y 2D. Las expresiones indican que E[R] crece proporcionalmente a √M; no debe confundirse E[R] con $\sqrt{E[R^2]} = \sqrt{M}$. Por ejemplo, para M = 1000 se espera E[R] ≈ 25,23 en 1D y ≈ 28,02 en 2D, frente a √M ≈ 31,62.

1.2 Objetivos

- Implementar en C y MPI las caminatas 1D y 2D, con parámetros configurables de pasos M, caminatas N y cantidad de procesos p.
- Contrastar la distancia promedio y la distribución final con las predicciones teóricas mediante simulación Monte Carlo.

- Medir speedup, eficiencia y tiempos de ejecución usando todas las cantidades consecutivas de procesos disponibles en cada equipo.
- Comparar los resultados entre cuatro sistemas y distinguir conclusiones observadas de hipótesis sobre el hardware.
- Como trabajo adicional, generalizar el programa a D dimensiones y comprobar los casos 3D y 4D.

2. Datos, implementación y metodología

El simulador genera las caminatas en memoria, sin datos de entrada externos. Cada ejecución informa la distancia promedio simulada y teórica, el error relativo y el tiempo de ejecución. La opción --sample permite guardar en CSV hasta 50.000 posiciones finales generadas por el proceso de rango 0 que se usa para los histogramas, evitando escribir los millones de trayectorias simuladas durante los benchmarks.

2.1 Configuración de los experimentos

Tabla 1. Sistemas utilizados.

	Clúster	**PC 6 núcleos**	**Netbook 4**	**PC 2 núcleos / SMT**
Procesador	AMD EPYC 7B12	Intel Core i5-9400	AMD Ryzen 3 7320U	Intel Core i7-7500U
Núcleos / hilos	Cuota de 12 CPU-equivalentes	6 núcleos físicos, 6 hilos	4 núcleos físicos, 8 hilos	2 núcleos físicos, 4 hilos lógicos
Entorno	Linux en contenedor	WSL2 (Windows)	Fedora Linux	WSL2 (Windows)
Procesos usados	1 a 12	1 a 6	1 a 4	1 a 4

En el clúster, cpu.max = 1200000 100000 corresponde a una cuota de 12 CPU equivalentes; no demuestra la exclusividad de 12 núcleos físicos.

Se realizaron tres tipos de pruebas: validación del modelo (N = 1.000.000, M = 100, 500, 1000, 2000 y 5000); histogramas 1D y 2D (M = 1000 y muestra de hasta 50.000 posiciones); y escalabilidad fuerte (N = 10.000.000, M = 1000, con p desde 1 hasta el máximo de cada sistema). Para la extensión 3D y 4D también se ejecutaron las validaciones y los benchmarks.

El conjunto de escalabilidad incluyó 26 configuraciones de procesos por dimensión (12 + 6 + 4 + 4). Sumadas las cuatro dimensiones, se obtuvieron 104 configuraciones y 520 mediciones individuales, con cinco repeticiones por configuración.

2.2 Estrategia de paralelización

La paralelización se realizó a nivel de caminatas completas y no dividiendo los pasos de una misma trayectoria. Esta decisión de diseño responde a que cada caminata posee una fuerte dependencia interna (la posición del paso k se obtiene obligatoriamente a partir del paso $k-1$), lo que requeriría comunicación constante de resultados parciales si se fragmentara. Por el

contrario, existe una total independencia entre las distintas caminatas, ya que calcular la distancia de una trayectoria no interfiere con las demás. Con un volumen de N = 10.000.000 de tareas de 1.000 pasos cada una, hay trabajo de sobra para ser distribuido entre los 12 procesos sin necesidad de segmentar ninguna caminata.

La distribución del trabajo se gestiona mediante un reparto estático: con N caminatas y p procesos, cada rango recibe $\lfloor N/p \rfloor$ caminatas, y los primeros N mod p procesos absorben una adicional. Dado que todas las caminatas constan exactamente de M pasos y conllevan el mismo costo computacional, la carga resulta sumamente equilibrada, con un desequilibrio máximo de apenas una caminata. Finalmente, el esquema requiere una comunicación mínima, limitándose a sincronizar los parámetros al inicio y reducir o sumar los resultados al terminar el procesamiento. Gracias a esta ausencia de dependencias entre procesos y al bajo costo de comunicación, el problema califica como prácticamente embarazosamente paralelo.

2.3 Implementación MPI y generación aleatoria

En cada paso se sortea una de las 2D direcciones, que determina el eje y el signo, y se suma ±1 en ese eje. Al terminar los M pasos se calcula $R = \sqrt{(x_1^2 + \ldots + x_D^2)}$ y se suma a un acumulador local. No se guarda la trayectoria, solo la posición actual. Como cada paso modifica una única coordenada, el costo es proporcional a N·M y no depende de D.

Se usa un selector acotado basado en multiplicación y rechazo, que evita el sesgo del operador módulo y calcula la constante de rechazo una sola vez al iniciar. Las versiones 1D y 2D son versiones especializadas del mismo esquema.

Se usa el generador PCG32, implementado dentro del código C. La semilla base es fija y cada proceso elige un flujo (*stream*) distinto a partir de su rango MPI, de modo que cada proceso tiene su propia secuencia privada sin comunicarse con los demás:

```
pcg32_srandom_r(&rng, base_seed, base_stream + (uint64_t)rank);
```

Esto da reproducibilidad: con los mismos parámetros y el mismo número de procesos se repiten las secuencias. Al cambiar p cambia el reparto de caminatas entre procesos, por lo que las trayectorias no son idénticas, pero sí las propiedades estadísticas. Por la misma razón, las cinco repeticiones de cada configuración usan las mismas semillas: sirven para medir la variabilidad del tiempo, no son cinco muestras estadísticamente independientes.

Se evaluaron tres técnicas para separar las secuencias por proceso: semillas separadas linealmente ($S_r = S_0 + r \cdot B$), flujos múltiples de PCG32 y *leapfrog* (saltos por bloques). La primera versión usó semillas lineales, y durante sus pruebas apareció un desbordamiento (descrito por el equipo como «desborde del búfer») que llevó a abandonarla. Se adoptó PCG32 con flujos por proceso porque es liviano, no requiere saltos de secuencia ni comunicación, y garantiza un flujo distinto por rango.

Las principales operaciones MPI fueron MPI_Bcast (parámetros), MPI_Barrier (sincronización previa al cronómetro) y MPI_Reduce, con MPI_SUM para la suma de distancias y MPI_MAX para seleccionar el tiempo local máximo. El proceso 0 informa el resultado final.

2.4 Medición y reproducibilidad

El tiempo se mide con MPI_Wtime, desde que todos los procesos pasan la barrera hasta que termina la suma global de distancias. Cada proceso mide su tiempo y se informa el máximo (MPI_MAX), es decir, el del proceso más lento. No incluye el arranque de mpirun. Todos los

tiempos informados son la mediana de cinco repeticiones, para reducir el efecto de picos ocasionales del sistema.

Se utilizan las métricas S(p) = T(1)/T(p) (speedup) y E(p) = S(p)/p (eficiencia). La referencia T(1) es siempre la ejecución del propio binario MPI con p = 1 en la misma máquina. Las versiones secuenciales compiladas con -O0 no se usan de referencia porque las variantes MPI se compilaron con -O3 -march=native.

Siguiendo el modelo de la cátedra, el tiempo global puede descomponerse en cálculo, comunicación y espera. Para esta aplicación se espera que predomine el cálculo; no obstante, no se registraron por separado estos componentes, por lo que la identificación de causas de pérdida de eficiencia requiere nuevas mediciones.

El proyecto automatiza con scripts/benchmark.sh, el Makefile y scripts/generar_graficos.py (pandas, NumPy y Matplotlib). Se comprobó que las medianas recalculadas desde los registros crudos coinciden con los CSV resumidos en las 104 configuraciones. Un ejemplo de ejecución en el clúster es:

```
PROCS="1 2 3 4 5 6 7 8 9 10 11 12" RUNS=5 \
MPI_FLAGS="--bind-to core --map-by core" \
make benchmark DIMS="3 4"
```

3. Resultados

3.1 Validación de 1D y 2D

La Tabla 2 presenta los resultados de las dos dimensiones centrales de la consigna, obtenidos en el clúster para N = 1.000.000 caminatas y M = 1000 pasos. En ambos casos la distancia promedio simulada coincide estrechamente con el valor aproximado teórico.

Tabla 2. Distancia promedio simulada y teórica (clúster, N = 1.000.000, M = 1000).

Dimensión	Distancia simulada	Distancia teórica	Error relativo
1D	25,2356	25,2313	0,017 %
2D	28,0305	28,0250	0,020 %

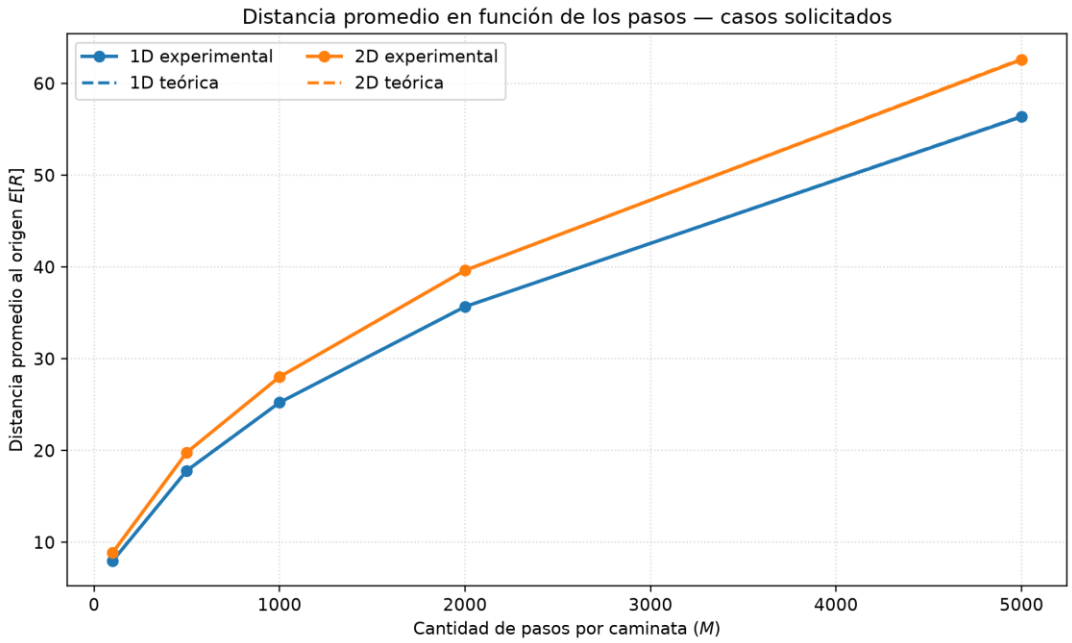


Figura 1. Distancia promedio al origen en función de la cantidad de pasos (1D y 2D). Los puntos son la simulación y las líneas punteadas la teoría.

En el rango M = 100 a 5000, las trayectorias medias siguen el crecimiento proporcional a √M. El mayor error relativo observado entre las 20 combinaciones validadas (incluidas las extensiones) fue 0,157 %. Este valor es pequeño frente a las distancias medidas, pero no implica exactitud determinista: las realizaciones de Monte Carlo presentan fluctuaciones estadísticas y las fórmulas son asintóticas para M grande.

3.2 Distribución de posiciones finales

En 1D, el histograma de posiciones finales para M = 1000 es aproximadamente simétrico alrededor de cero y sigue la forma normal predicha para un paseo sin deriva; su desviación típica teórica es √1000 ≈ 31,6.

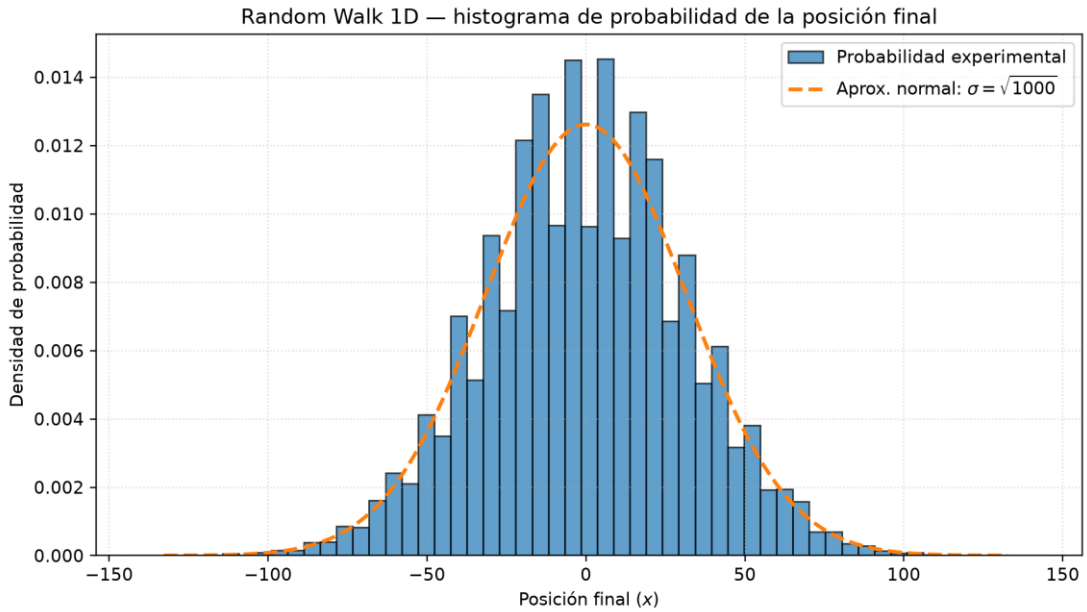


Figura 2. Histograma de probabilidad de la posición final en 1D (M = 1000) y curva normal teórica.

La alternancia de barras altas y bajas no indica un defecto del simulador. Como M = 1000 es par, solo pueden alcanzarse posiciones pares. Los intervalos del histograma tienen un ancho cercano a cinco unidades y contienen alternadamente dos o tres valores permitidos.

En 2D el color indica la densidad de probabilidad de terminar en cada celda del plano. Es más intenso en el origen y se atenúa por igual en todas las direcciones, es decir que ningún eje está privilegiado. La circunferencia punteada tiene radio igual a la distancia promedio teórica, E[R] ≈ 28,02.

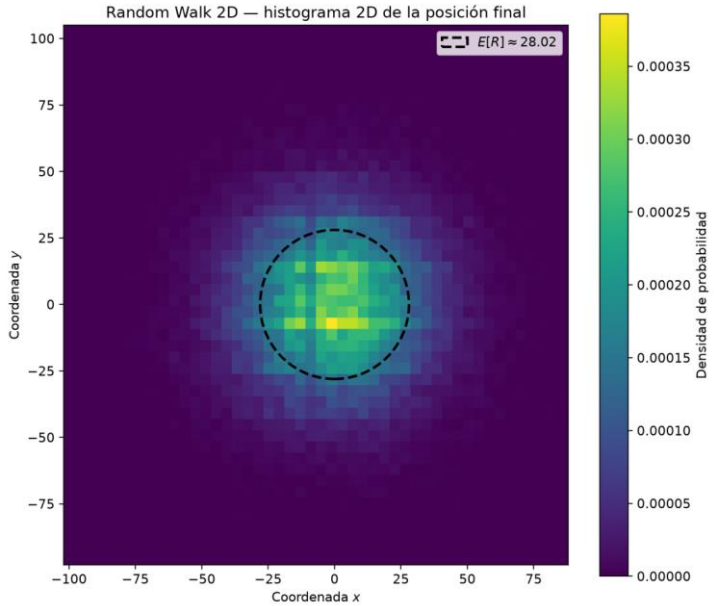


Figura 3. Histograma bidimensional de la posición final en 2D (M = 1000). La circunferencia tiene radio E[R].

3.3 Escalabilidad en el clúster

Con N = 10.000.000 y M = 1000 fijos (strong scaling), las cuatro dimensiones se aceleran casi linealmente hasta p = 12. La Tabla 3 muestra las cifras principales del proyecto.

Tabla 3. Escalabilidad en el clúster con p = 12 (N = 10^7, M = 1000).

Dimensión	T(1) [s]	T(12) [s]	Speedup S(12)	Eficiencia E(12)
1D	14,351	1,198	11,978	99,82 %
2D	65,785	5,484	11,996	99,97 %

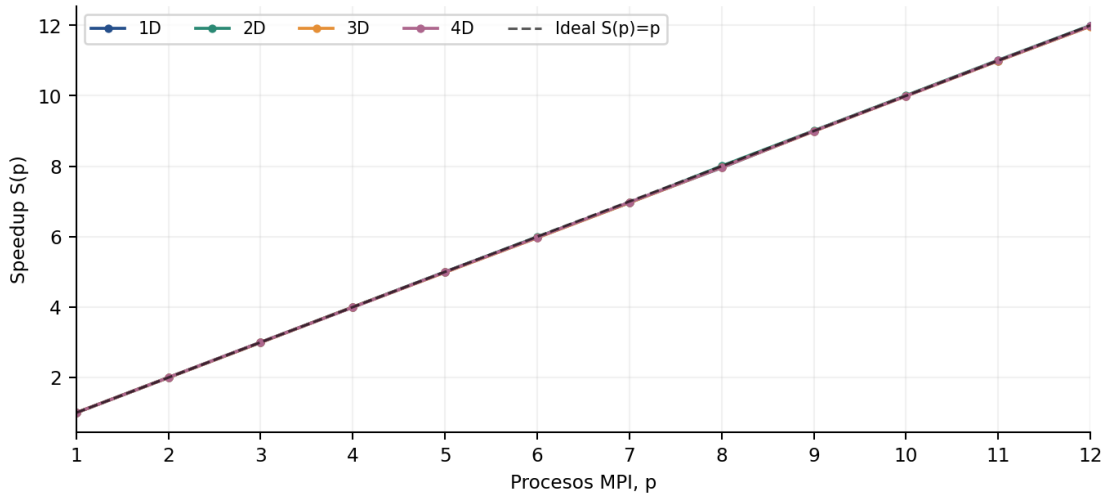


Figura 4. Speedup en el clúster. Se muestran 1D y 2D junto con las curvas casi superpuestas de las extensiones 3D y 4D.

El trabajo independiente entre procesos y el reducido volumen de datos comunicados son compatibles con este comportamiento casi ideal. Sin embargo, no se midieron por separado los tiempos de comunicación y no corresponde extrapolar los resultados a más de 12 procesos

La figura 5 muestra E(p) = S(p)/p en porcentaje. El eje vertical va solo de 98 % a 101 %, así que cada división vale 0,5 puntos y cualquier diferencia se ve ampliada.

Todas las curvas están entre 99,5 % y 100,1 %, con un leve descenso entre p = 6 y p = 8 que se recupera después. Una variación menor a 0,6 puntos, que sube y baja sin tendencia, es ruido de medición y no pérdida de escalabilidad. Lo importante es que la eficiencia no cae al aumentar p. Los valores apenas sobre 100 % (hasta 100,1 %, en 2D) se interpretan como variación de medición y no como superlinealidad.

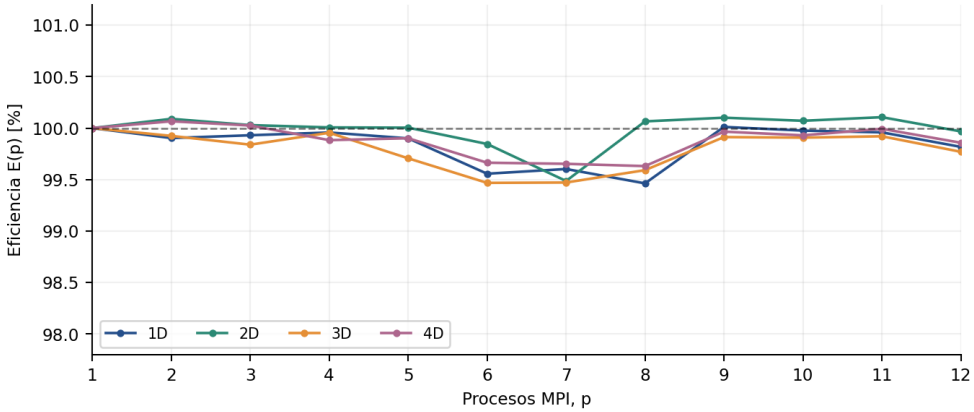


Figura 5. Eficiencia paralela en el clúster. El eje vertical abarca solo de 98 a 101 %.

3.4 Comparación entre los cuatro sistemas

La comparación entre máquinas requiere distinguir eficiencia y velocidad absoluta. En la Tabla 4 se informa la aceleración con el mayor número de procesos que se probó en cada entorno; los máximos de p no son equivalentes entre equipos.

Tabla 4. Speedup con el mayor número de procesos de cada sistema ($N = 10^7$, $M = 1000$).

Sistema (p máximo)	1D: S(p)	2D: S(p)	1D: E(p)	2D: E(p)
Clúster (12)	11,978	11,996	99,82 %	99,97 %
PC 6 núcleos (6)	5,162	5,714	86,03 %	95,23 %
Netbook 4 (4)	3,674	3,646	91,84 %	91,16 %
PC 2 núcleos / SMT (4)	2,257	2,821	56,43 %	70,53 %

Para una comparación a igual cantidad de procesos, se considera p = 4, presente en los cuatro sistemas. El clúster se mantiene cerca del 100 %; la PC de seis núcleos ronda el 97 %; la netbook alcanza cerca del 91 %; y la PC con dos núcleos físicos pierde eficiencia nominal al asignar cuatro procesos a cuatro hilos lógicos, que no equivalen a cuatro núcleos independientes.

Tabla 5. Eficiencia con p = 4 procesos.

Sistema	1D	2D
Clúster	99,96 %	100,01 %
PC 6 núcleos	97,01 %	97,60 %
Netbook 4	91,84 %	91,16 %
PC 2 núcleos / SMT	56,43 %	70,53 %

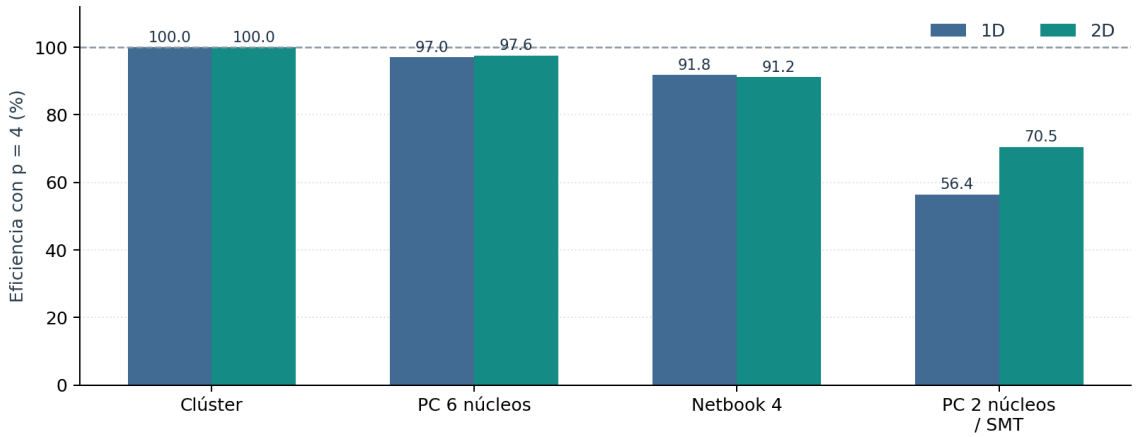


Figura 6. Comparación de eficiencia con cuatro procesos: únicamente los casos 1D y 2D.

Las diferencias son compatibles con efectos de frecuencia, ocupación del sistema operativo, límites de recursos y topología de núcleos e hilos (El programa y los parámetros son los mismos, por lo que se piensa que la diferencia viene del equipo). No se controlaron ni midieron todas estas variables, por lo que deben tratarse como hipótesis de interpretación, no como causas demostradas.

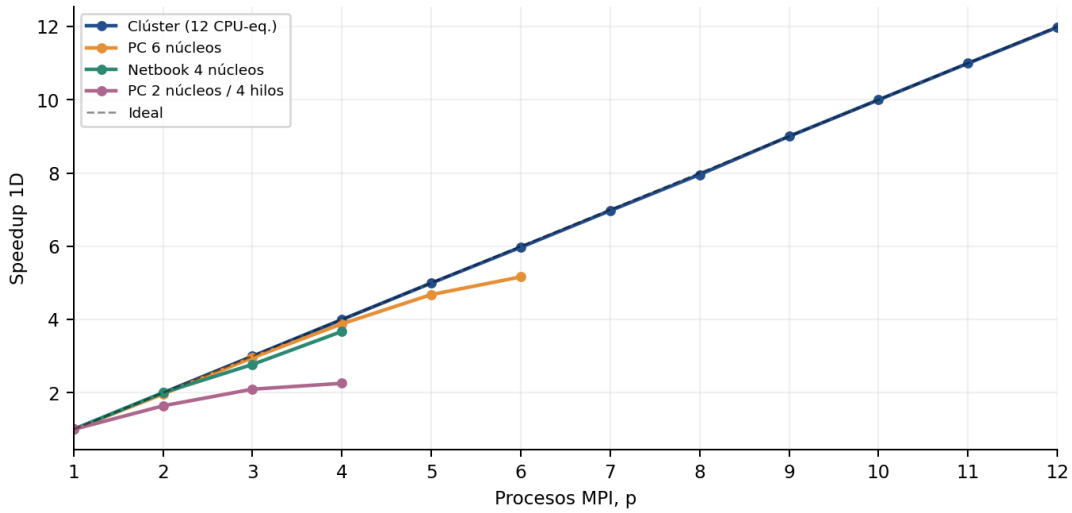


Figura 7. Evolución del speedup 1D para cada sistema; cada serie termina en su p máximo.

La Figura 7 muestra la evolución del speedup en 1D con todos los valores de p. Cada curva termina donde termina el máximo de procesos de ese equipo, por lo que las curvas más cortas no son peores, solo tienen menos puntos medidos.

3.5 Tiempo absoluto y anomalía de 2D

Una eficiencia más alta no garantiza un menor tiempo de ejecución: el speedup compara un sistema consigo mismo. Por ejemplo, en 1D con p = 4 la netbook tarda 3,097 s, la PC de 6 núcleos 3,440 s y el clúster 3,589 s, aunque el clúster sea el de mejor eficiencia.

La Tabla 6 presenta los tiempos con p = 4 de las dos variantes requeridas.

Tabla 6. Tiempos medianos ($N = 10^7$, $M = 1000$).

Sistema	T(1) 1D [s]	T(4) 1D [s]	T(4) 2D [s]
Clúster	14,351	3,589	16,445
PC 6 núcleos	13,348	3,440	15,382
Netbook 4	11,378	3,097	14,210
PC 2 núcleos / SMT	15,289	6,773	24,362

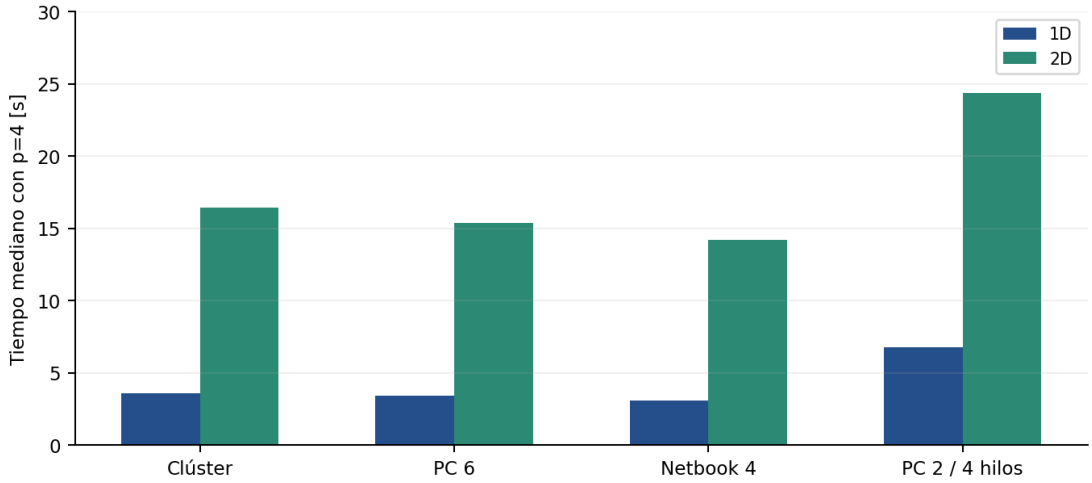


Figura 8. Tiempos medianos para p = 4: 1D frente a 2D en cada sistema.

La netbook resuelve 1D más rápido que el clúster con p = 4 (3,097 s frente a 3,589 s), aunque su eficiencia es inferior. El clúster resulta más rápido al poder utilizar hasta 12 procesos (1,198 s en 1D). Por ello, la escalabilidad no equivale a una clasificación absoluta de CPU.

Tabla 7. Costo por paso en el clúster con un proceso.

Dimensión	T(1) [s]	Costo por paso [ns]	Relativo a 1D
1D	14,351	1,44	1,00
2D	65,785	6,58	4,58
3D	17,807	1,78	1,24
4D	17,794	1,78	1,24

La implementación 2D destaca por un costo mayor: en el clúster, con un proceso, demora 65,785 s frente a 14,351 s en 1D. A igual $N \cdot M = 10^{10}$ pasos, equivale a 6,58 ns por paso en 2D y 1,44 ns en 1D. Como referencia interna, las ejecuciones de la versión generalizada en 3D y 4D rondan 1,78 ns por paso, es decir que 2D tarda alrededor de 3,7 veces más que 3D pese a tener menos dimensiones.

Una explicación posible, todavía no verificada, es que el programa 2D es más lento por paso porque elige la dirección con condicionales (if/else) que el procesador predice mal, mientras que random_walk_nd.c elige la dirección sin condicionales (multiplicación por la cota y desplazamiento de bits). Para verificarla habría que medir random_walk_nd con D = 2 en el mismo entorno y realizar perfiles de CPU.

3.6 PC de dos núcleos: hilos lógicos

El i7-7500U posee dos núcleos físicos y cuatro hilos lógicos (SMT/Hyper-Threading). Las ejecuciones con p = 4 utilizaron --oversubscribe --bind-to none. Cuatro procesos superan el número de núcleos físicos, pero no el número de CPU lógicas; no se estudió p > 4 ni se fijó la afinidad.

Tabla 8. Aceleración y eficiencia nominal en la PC con SMT

Dimensión	S(4)	E(4) = S(4)/4
1D	2,257	56,4 %
2D	2,821	70,5 %

La eficiencia E(4) compara contra cuatro unidades ideales de cómputo y, por tanto, no equivale a la utilización física de cuatro núcleos. El cociente S(4)/2 puede servir como referencia descriptiva frente a dos núcleos ideales, pero no mide directamente la ganancia de SMT. Para estimar la mejora observada al pasar de dos a cuatro procesos se necesita T(2)/T(4) = S(4)/S(2), y para atribuirla específicamente a SMT se requieren afinidad y condiciones de frecuencia controladas.

3.7 Extensión a 3D y 4D

Como mejora, se implementó un simulador generalizado a D dimensiones y se probaron 3D y 4D. La extensión conserva la estrategia de reparto por caminatas y las operaciones colectivas MPI; únicamente cambia el número de coordenadas y direcciones posibles.

Tabla 9. Síntesis de los resultados adicionales de 3D y 4D.

Caso	R² simulada	E[R] teórica	Error (M = 1000)	S(12) clúster	E(12)
3D	29,1369	29,1346	0,008 %	11,972	99,77 %
4D	29,7570	29,7250	0,108 %	11,983	99,86 %

Estos resultados documentan que la generalización se ejecutó y conservó una buena escalabilidad para el tamaño de problema ensayado. No se realiza aquí un estudio estadístico detallado de 3D y 4D: el foco de validación, las comparaciones gráficas y la discusión principal corresponden a 1D y 2D. Los errores observados en una muestra de Monte Carlo no sustituyen un análisis entre múltiples semillas independientes.

4. Conclusiones

Se implementó y validó un simulador MPI de Random Walk 1D y 2D, de acuerdo con el alcance principal del proyecto. Las distancias promedio concuerdan con la teoría, crecen aproximadamente como $\sqrt{M}$ y los histogramas muestran las distribuciones esperadas: una campana centrada en cero en 1D y simetría aproximadamente circular en 2D.

La estrategia de distribuir caminatas completas resultó adecuada. En el clúster se obtuvo un speedup de 11,978 (1D) y 11,996 (2D) con 12 procesos, ambos próximos al ideal. En las computadoras personales, los resultados fueron más variables. Las mediciones respaldan que la aplicación tiene poca necesidad de comunicación, pero no permiten separar de forma causal los efectos de hardware, planificación y frecuencia.

La evaluación también mostró que speedup y tiempo absoluto responden preguntas distintas. Además, la versión especializada de 2D resultó considerablemente más lenta por paso que la generalizada usada como referencia. Este hallazgo señala una mejora concreta que debe evaluarse mediante comparación de binarios y perfiles.

Finalmente, la generalización a 3D y 4D constituye una contribución adicional del equipo: muestra la reutilización de la arquitectura MPI y permite explorar nuevas dimensiones sin cambiar el esquema básico de distribución del trabajo.

4.1 Limitaciones

- La escalabilidad fuerte se midió con un único tamaño (N = 10.000.000, M = 1000). No se realizó weak scaling ni un barrido de tamaños menores.
- El clúster fue un contenedor con cuota de 12 CPU equivalentes en un solo nodo; no se evaluó comunicación entre nodos ni escalamiento por encima de doce procesos.
- Las cinco repeticiones por configuración usan semillas fijas: reflejan variación temporal y no incertidumbre estadística entre realizaciones independientes.
- No se registraron tiempos locales por proceso ni costos desglosados de cálculo, comunicación y espera. Tampoco se monitorizaron frecuencia de CPU o fallos de predicción.
- Los entornos son heterogéneos (Fedora, WSL2 y Linux en contenedor), por lo que los tiempos absolutos no son una comparación controlada de procesadores.
- No se fijó la afinidad de la PC de dos núcleos ni se ensayó p > 4; la independencia estadística entre streams de PCG32 no fue demostrada formalmente.
- El código del clúster y de la netbook fue contrastado como idéntico; de las otras dos computadoras se conservaron los resultados, pero no el código fuente completo empleado.

4.2 Mejoras recomendadas

Se priorizan cuatro líneas de mejora que amplían directamente la validez de los resultados y la calidad de la implementación:

1. Explorar más tamaños de problema. Variar N y M para determinar cuándo deja de convenir aumentar p; agregar pruebas de weak scaling, manteniendo el trabajo por proceso aproximadamente constante.
2. Optimizar la implementación 2D. Registrar tiempos de cálculo, comunicación y espera por proceso, además de frecuencia y eventos relevantes de CPU; comparar la versión 2D especializada con random_walk_nd en D = 2 antes de cambiar el algoritmo.
3. Fortalecer la validación estadística. Repetir las pruebas principales con varias semillas base independientes, reportar media y error estándar, y documentar el esquema de inicialización de streams del generador.
4. Explorar una implementación híbrida que combine MPI y OpenMP para aprovechar las ventajas de ambos modelos de programación paralela. MPI permitiría distribuir las caminatas entre procesos, mientras que OpenMP facilitaría la ejecución concurrente mediante múltiples hilos dentro de cada proceso.

5. Referencias

1. Material de la cátedra: Computación de Alto Rendimiento para aplicaciones numéricas. Métodos Numéricos II, FACET-UNT (2026).

2. MPI Forum. MPI: A Message-Passing Interface Standard. https://www.mpi-forum.org/docs/

3. Open MPI Project. Open MPI Documentation. https://docs.open-mpi.org/

4. O'Neill, M. E. (2014). PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. https://www.pcg-random.org/paper.html

5. O'Neill, M. E. Using the Full C Implementation - PCG. https://www.pcg-random.org/using-pcg-c.html

6. Free Software Foundation. GNU Make Manual. https://www.gnu.org/software/make/manual/make.html

Link al repositorio en github